

Lecture 7

- Responsive design
 - grids
 - media queries
 - front-end frameworks
 - Bootstrap
- Accessibility
 - general guidelines
 - ARIA labels
 - tools



Responsive design



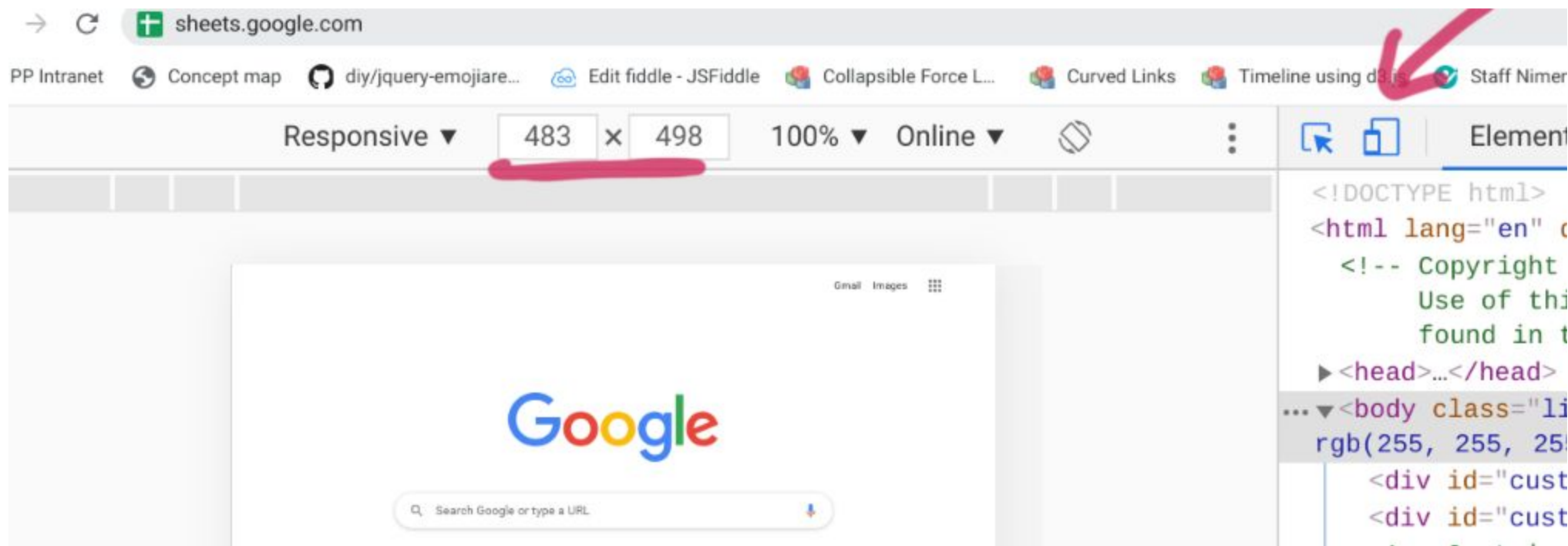
makes web pages functional and good-looking on a variety of devices and screen sizes.

Responsiveness with frontend frameworks

- Implementing layouts with CSS used to be laborious:
 - CSS flex, grid and other new properties improve this a lot
- Front-end frameworks
 - Bootstrap, Foundation, Semantic UI, Materialize, Material UI...
- Web development frameworks
 - transfer design as single-page applications (SPA)
 - manage also code and business logic
 - React, Vue, Angular, Laravel, Elm
 - more of web technologies in [Advanced Frontend Technologies](#)
 - React in COMP.CS.210 (Principles of Programming Graphical User Interfaces)

Developer tools to test different screen sizes

Open Toggle Device Toolbar (Ctrl-shift-M) in Chrome



Responsive design with grids

fraction *fr* is the first prompt towards responsiveness, but *fr* alone does no trick, there are three more steps to take:

1. repeat(N, width)
2. minmax(min, max)
3. auto-fill/auto-fit as the multiplier N

1. ^{multiplier}repeat(N, width)

Remember grid-template-columns and the width definitions:

```
grid-template-columns: 1fr 1f 1fr 250px;
```

and removing the tautology with repeat:

```
grid-template-columns: repeat(3, 1fr) 250;
```

The 1st parameter (3) is a multiplier, the 2nd, (1fr) width.

Either or both of these parameters can be made more flexible.

2. minmax(min, max)

Width can be defined as minmax() range that attempts to guarantee a decent space for the content with min.

Here, as a mix of absolute and fraction values: **minmax(100px, 1fr)**

```
grid-template-columns: repeat(3, minmax(100px, 1fr)) 250px;
```

- if $1fr > 100px$, the width is 1fr
- if $1fr < 100px$, the width is min-width: 100px; // the rest is taken from 250px
 - a. The last column, 250px is used as a “buffer” taking what is left from $3 * minmax()$.

Based on the chosen strategy, elements may overflow or wrap, and if space is very limited, also the min width may be compromised.

3. auto-fill vs. auto-fit

In bad design, elements start to overflow. In the worst case, they are not visible anymore.

To make a grid container responsive, the first repeat() parameter, the multiplier N, must also be "gridified" in addition to minmax() of length

For the purpose, there is the last step: auto-fill vs auto-fit.

```
grid-template-columns: repeat(auto-fill, minmax(100px, 1fr)) 250px;
```

```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr)) 250px;
```


auto-fill vs. -fit differ if there is space to grow

If `minmax(100px, 1fr)`:

auto-fill: min of 100px is used,

auto-fit: max of 1fr is used instead.

Responsive grid: mnemonic

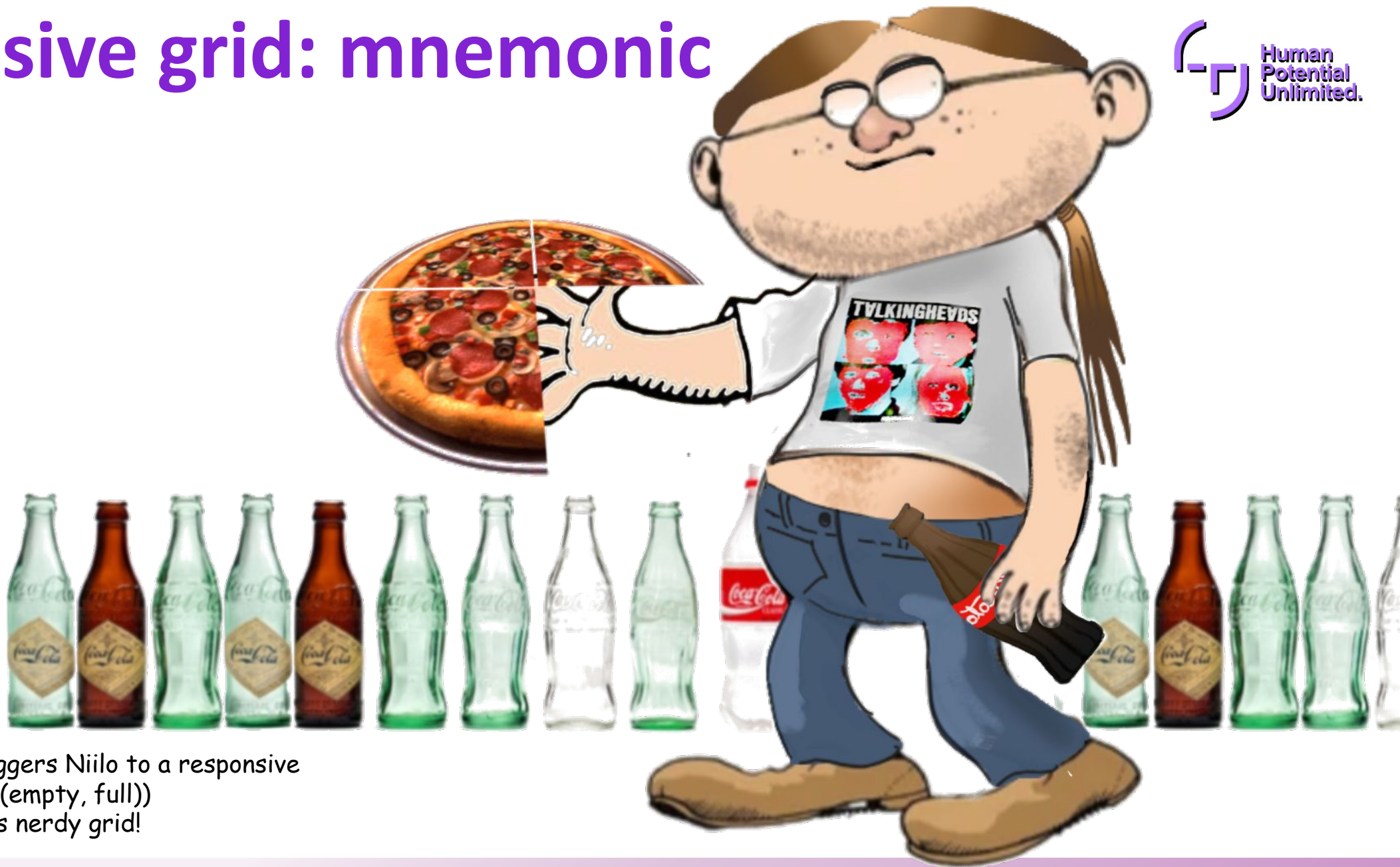
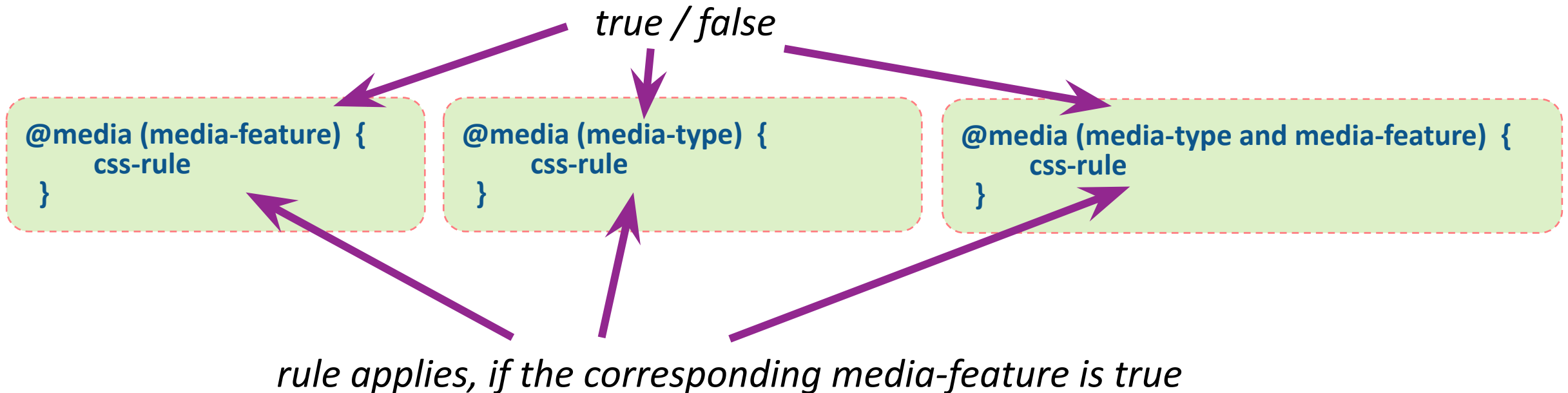


Fig. A pizza fraction triggers Niilo to a responsive
`repeat(auto-fill, minmax(empty, full))`
- Niilo cannot live off his nerdy grid!

Media queries

- Adapting to different devices with CSS3 media queries



Media query: media-features

○ Media-features:

width | min-width | max-width | height |
min-height | max-height | aspect-ratio | color
| resolution | orientation | scan ...

```
@media (min-width: 768px) { css-rule }
```

```
@media (min-width: 768px) and (max-width: 991px) { css-rule }
```

Media query: media-type

○ Media-types:

all | aural | braille | handheld | print |
projection | screen | tty | tv | embossed |
speech ..

```
@media (print) { css-rule }
```

```
@media (min-width: 700px), handheld and (orientation: landscape)  
{ css-rule }
```

Typical media-query

- Basic style rules and special style rules for certain screen widths
 - first we define the rules that apply regardless of the window width
 - with media queries we change the basic style rules, or add new rules for certain situations

```
p {  
  color: blue;  
}  
@media (min-width: 1200px) {  
  rules for wide screens only  
}  
@media (min-width: 992px) and (max-width: 1199px) {  
  rules for laptop sized screens  
}
```

...

*Non-overlapping
widths*

Bootstrap

<http://getbootstrap.com/>



Build responsive, mobile-first projects on the web with the world's most popular front-end component library.

Bootstrap is an open source toolkit for developing with HTML, CSS, and JS. Quickly prototype your ideas or build your entire app with

our Sass variables and mixins, responsive grid system, extensive prebuilt components, and powerful plugins built on jQuery.

- Most popular? The third most starred in GitHub
- Framework takes advantage of CSS-classes: html elements are assigned with certain class values, or certain html-structures are provided ready made

- Mobile first?



or



Using Bootstrap

Starter template

Bootstrap

Build responsive, mobile-first projects on the web with the world's most popular front-end component library.

Bootstrap is an open source toolkit for developing with HTML, CSS, and JS. Quickly prototype your ideas or build your entire app with our Sass variables and mixins, responsive grid system, extensive prebuilt components, and powerful plugins built on jQuery.

[Get started](#)
[Download](#)

Currently v4.3.1



```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
```

```
  <meta charset="utf-8">
```

```
  <!-- needed for responsive mobile (small) device browsing -->
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <title>Using Bootstrap</title>
```

```
  <!-- Bootstrap CSS -->
```

```
  <link rel="stylesheet" href="css/bootstrap.css">
```

```
</head>
```

```
<body>
```

```
  <!-- The whole page is typically defined to have the class value "container" -->
```

```
  <main class="container">
```

```
    <h1>Bootstrap, jippi jai jei!</h1>
```

```
  </main>
```

```
  <!-- jQuery JavaScript: needed to implement functionality of components, v 4.3. is the last -->
```

```
  <script src="js/jquery-3.1.min.js"></script>    <!-- stable version at the moment -->
```

```
  <!-- Bootstrap JavaScript: Bootstrap contains also functionality, not only styles -->
```

```
  <script src="js/bootstrap.bundle.js"></script>    <!-- bundle-version, to be safe -->
```

```
  <script src="js/oma.js"></script>    <!-- if you have own scripts to include -->
```

```
</body>
```

```
</html>
```

JWT

01

Bootstrap template

JWT

02

Bootstrap minimized

Nice, predefined styles for basic elements

- Taken into use by giving the element certain class values
- For example the buttons <https://getbootstrap.com/docs/4.1/components/buttons/>

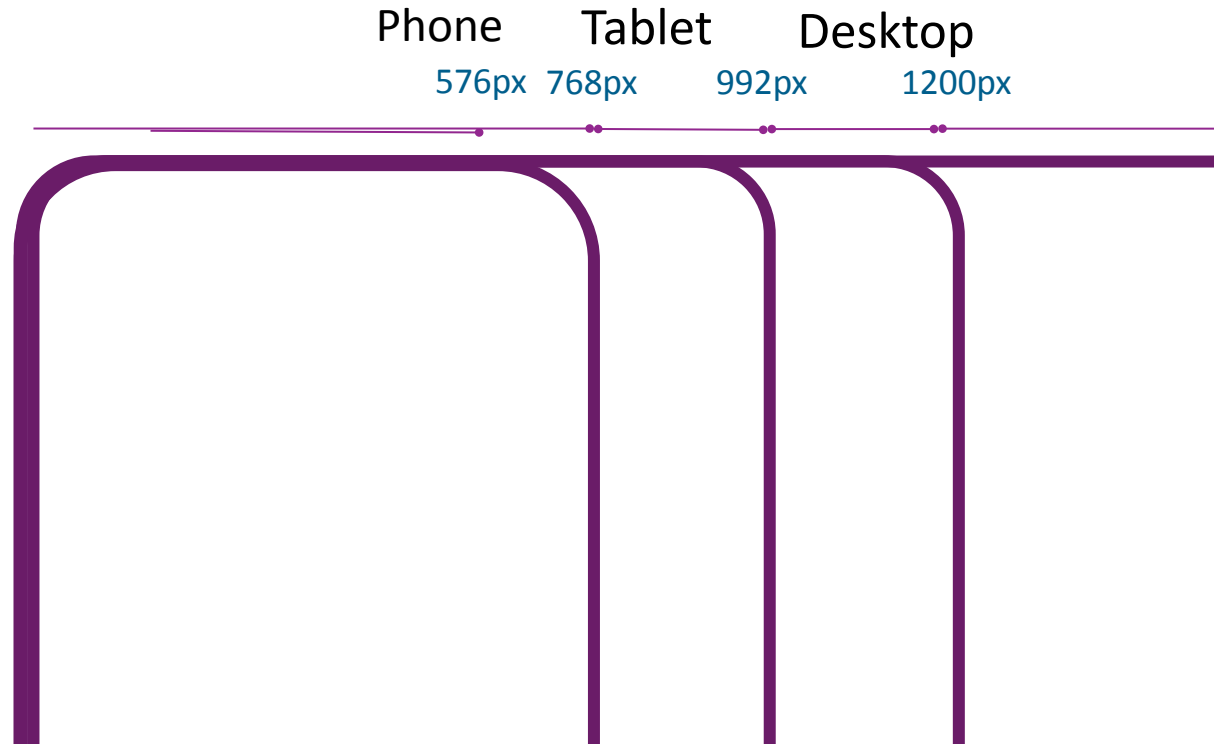


```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-info">Info</button>
<button type="button" class="btn btn-light">Light</button>
<button type="button" class="btn btn-dark">Dark</button>
<button type="button" class="btn btn-link">Link</button>
```

Using Bootstrap grid for flexible layouts



Typical viewport widths



- Small phone
< 576px
- Phone
< 576px–768px
- Tablet
768px – 992px
- Desktop
992px – 1200px
- Wide desktop
> 1200px

Bootstrap container (always 12 columns)

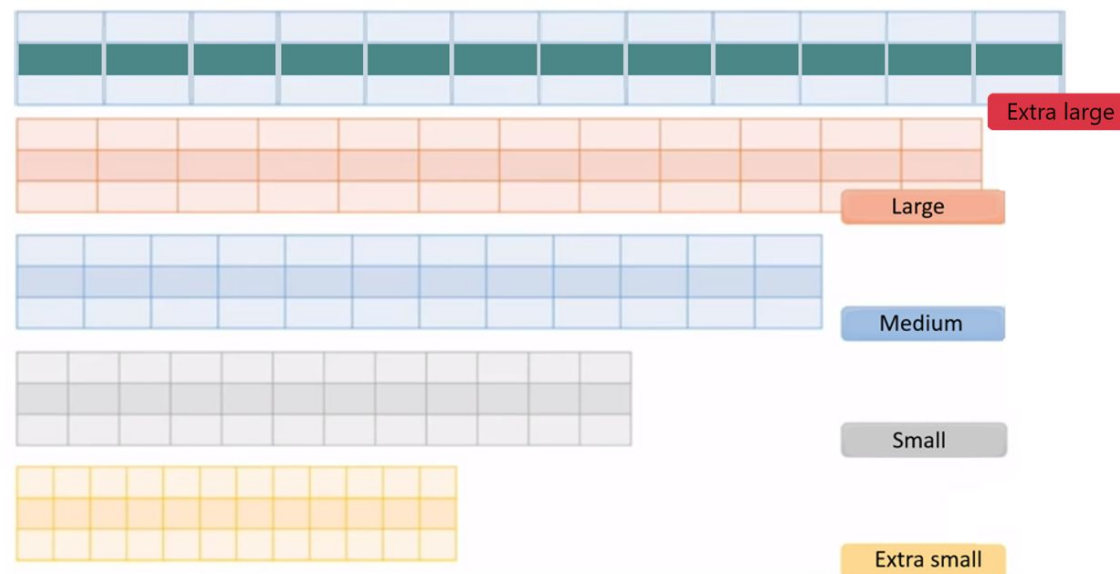
```
<div class="container">  
  ...  
</div>
```

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-
# of columns	12				
Gutter width	30px (15px on each side of a column)				
Nestable	Yes				
Column ordering	Yes				

Bootstrap grid (i.e., flex of flexes)

- Implemented using Flexbox
 - uses mostly `rem`s or `em`s , but for grid breakpoints and container widths `px`s are used
 - 12 column grid
- You can think that there are five containers (each divided into 12 columns)
- One for each device width
 - `xl`: wider than 1200px :
extra large screens
 - `lg`: 992px-1199:
desktop (large)
 - `md`: 768px - 991 px
laptop (medium)
 - `sm`: 576px - 767px
tablet (small)
 - `xs` : less than 576px
phone (extra small)

Bootstrap grid



Bootstrap grid usage

- 12 columns, 30px gutter
 - Bootstrap grid is defined like a `<table>` element
 - a row element (`class="row"`)
 - a cell element (`class="col"`)
 - The shorthands, `xs`, `sm`, `md`, `lg`, `xl`, are used to assign the element as a cell element only for certain display widths, for example:
 - `class="col-lg"`
- element is treated as a cell element only when display width is large or wider

```
<main class="container">
  <!-- layout Bootstrap grid-->
  <div class="row">      <!-- "row" -->
    <div class="col-sm">  <!-- if display sm -->
      <h3>FIRST</h3>    <!-- "column 33.33%" -->
      Lorem ipsum ...
    </div>
    <div class="col-sm">  <!-- "column 33.33%" -->
      <h3>SECOND</h3>
      Mauris sed ...
    </div>
    <div class="col-sm">  <!-- "column 33.33%" -->
      <h3>THIRD</h3>
      Vivamus placerat ...
    </div>
  </div>
</main> <!-- end of container -->
```

Hierarchical columns

- Columns can be defined hierarchically
 - inside a column, we define a row, which, then then is further divided into 12 columns

```
<div class="container">
  <div class = "row">
    <div class = "col-xs-6 col-md-4">
      <h3>First column that will be divided further</h3>
      <div class = "row">
        <div class= "col-md-6"><h3>Column 1A</h3></div>
        <div class= "col-md-6"><h3>Column 1B</h3></div>
      </div>
      <div class= "col-xs-6 col-md-4"><h3>Column 2A</h3></div>
      <div class= "col-xs-6 col-md-4"><h3>Column 2B</h3></div>
    </div>
  </div>
```


Responsive page using Bootstrap grid

- Column-element is given as a class: `col-deviceWidth-numberOfCols`
- Name consists of three parts:
 - reserved word `col`
 - deviceWidth of the viewport: `xs`, `sm`, `md`, `lg`, `xl`
 - each of the size refers to itself **and** wider
 - e.g. `.sm` refers to sizes *medium*, *large* and *extra large*
 - nbr of columns: `1-12`
 - e.g.
 - `col-lg-4` (the element uses four columns when the viewport width is *large*, or *extra large*)
 - `col-xl-2` (the element uses two columns when the viewport width is, if the element also had the previous class name this one overrides the size *large*)
 - default is, that all columns are used, so `xs`, `sm`, `md`, which were not defined above take the use of all the 12 columns

More responsiveness

- When we want a more responsive layout
 - with specified layouts for several display widths
 - the class values are added with the `nr` of grid cells the element occupies on each display width
 - shorthand “`col-minwidth-nr`”, e.g.

`col-12` = the default is that the element occupies the whole width of the `.container` element, which is 12 cells

`col-sm-6` = when the display `sm` or larger, the element occupies 6 cells

`col-md-4` = when the display `md` or larger, the element occupies 4 cells

`col-lg-3` = when the display `lg` or larger, the element occupies 3 cells

`col-xl-2` = when the display `xl` or larger, the element occupies 2 cells

Examples

- `<element class="col-sm-4">`
 - the element uses 4 columns when the width is small, medium, large, extra large (viewport width > 576px)
- `<element class="col-lg-2">`
 - the element uses 2 columns when viewport large or extra large
- `<element class="col-sm-4 col-lg-2">`
 - element has both classes => second one (2 columns) overrides the devices with large or extra large viewports (width > 992px)
 - viewport sizes `xs` and `sm` were not defined, so they use all the 12 columns (width < 768px)

An example

o How does this `div` behave?

```
<div class="col-xs-6 col-sm-4 col-md-2">Content</div>
```

- width `xs` uses 6 grid columns (layout in 2 columns)
- width `sm` uses 4 grid columns (layout in 3 columns)
- widths `md`, `lg` and `xl` use 2 grid columns (layout in 6 columns)

Hiding elements with bootstrap

○ Options: `.visible-devicewidth` / `.hidden-devicewidth`

- hidden
 - hidden as a default
- visible
 - visible as a default

○ For defining layout for printed version of the page:

`.hidden-print`
`.visible-print-block`
`.visible-print-inline`
`.hidden-print-inline-block`

```
<div class="jumbotron"> <!-- otsikkopalkki -->

    <h1 class="visible-lg">Large</h1>
    <h1 class="visible-md">Medium</h1>
    <h1 class="visible-sm">Small</h1>
    <h1 class="visible-xs">Extra small</h1>

</div>
```

Accessibility (a11y)

Write HTML with good accessibility in mind. Provide the user a good way to navigate and interact with your site: main things easily accessible

Make your HTML code as semantic as possible, so that the code is easy to understand for visitors and screen readers.

<https://vm.fi/saavutettavuusdirektiivi>

Semantic HTML

- Semantic `<button>Click Me</button>`
vs. Non-semantic `<div>Click Me</div>`
- Headings are important
- Alternative Texts are important too

Clear Language / Good Links

https://www.w3schools.com/html/html_accessibility.asp

WAI-ARIA

- o Web Accessibility Initiative – Accessible Rich Internet Applications defined in W3C
<https://www.w3.org/WAI/standards-guidelines/aria/>
- o WAI has come up with content accessibility guidelines WCAG 2.0:
<https://www.w3.org/WAI/standards-guidelines/wcag/>
- o Tools for guidelines compliance check:
 - VSCode extension for Web Accessibility:
maxvanderschee.web-accessibility
 - Plussa uses **axe devtools** in a11y grading, browser extensions available: [more about axe](#)

o Test with screenreaders

- NVDA <https://www.nvaccess.org/> (Windows, freeware)
- Jaws (Windows, commercial)
- VoiceOver (Apple)
- ChromeVox <chrome://settings/accessibility> (Text-to-speech)
- TalkBack (Android)
- Orca <https://help.gnome.org/users/orca/stable/> (Linux)
- Survey about the current screen readers:
<https://webaim.org/projects/screenreadersurvey8/>